Algorithmique

Cours et exemples

Un élève décrit oralement la construction d'une figure très simple sans la montrer ; un autre l'exécute. Comparer la figure obtenue à la figure cible et repérer les instructions ambiguës. Réécrire la procédure avec des données d'entrée, des instructions ordonnées et un résultat contrôlable.

Objectifs

- Comprendre la notion d'algorithme et ses propriétés.
- Lire, exécuter et tracer un algorithme en pseudo-code.
- Tracer et lire un organigramme.
- Écrire un algorithme simple pour un problème mathématique.

Qu'est-ce qu'un algorithme ?

Un **algorithme** est une suite finie et non ambiguë d'instructions permettant de résoudre un problème. Il doit être : **fini** (il termine), **valide** (il donne le bon résultat), et **déterministe** (même entrée $\Rightarrow$ même sortie).

Variables, affectation, séquence

Une **variable** est un emplacement mémoire identifié par un nom. L'instruction $A \leftarrow 5$ signifie affecter la valeur 5 à A . Attention : $A \leftarrow A + 1$ remplace l'ancienne valeur de A par (ancienne valeur) $+ 1$.

Variables, affectation, séquence

Exécuter : $A \leftarrow 3, \quad B \leftarrow A + 7, \quad A \leftarrow A \times 2$

Étape	Instruction	A	B
1	$A \leftarrow 3$	3	-
2	$B \leftarrow A + 7$	3	10
3	$A \leftarrow A \times 2$	6	10

Structures de contrôle

- **Séquence** : les instructions s'exécutent dans l'ordre.
- **Alternative** (`Si … Sinon`) : exécution conditionnelle.
- **Itération** (`Tant que` / `Pour`) : répétition.

Pour *exécuter à la main* un algorithme :

1. Créer un tableau avec une colonne par variable.
2. Exécuter chaque instruction dans l'ordre, ligne par ligne.
3. À chaque affectation, mettre à jour la colonne concernée en barrant l'ancienne valeur.
4. Pour un `Si`/`Sinon`, évaluer la condition *avant* de choisir la branche.
5. Pour un `Tant que`, répéter l'évaluation de la condition à chaque itération.

Méthodes et compléments de cours

Dans ce manuel, on utilise les conventions suivantes :

- **Affectation :** $X \leftarrow$ expression
- **Lecture :** `Lire`(X) (saisie utilisateur)
- **Écriture :** `Afficher`(X)
- **Alternative :** `Si` *condition* `alors` … \ `Sinon` … `Fin Si`
- **Boucle bornée :** `Pour` *i* `de` *a* `à` *b* `faire` … `Fin Pour`
- **Boucle non bornée :** `Tant que` *condition* `faire` … `Fin Tant que`

Un **invariant de boucle** est une propriété vraie avant la boucle, conservée à chaque itération, et qui permet de prouver la correction de l'algorithme à la sortie.

Le **coût** d'un algorithme se mesure en comptant le nombre d'opérations élémentaires effectuées en fonction de la taille n de l'entrée. Plus ce nombre croît lentement avec n, plus l'algorithme est efficace. On parle d'algorithme *rapide* (peu d'opérations) ou *lent* (beaucoup d'opérations).